

Docker Notes

1. Troubleshooting Commands

1.1 Check running containers

```
docker ps
```

1.2 Check all containers (including stopped)

```
docker ps -a
```

1.3 Check logs of a container

```
docker logs <container-name>
```

1.4 Follow logs (live)

```
docker logs -f <container-name>
```

1.5 Enter inside a running container (shell access)

```
docker exec -it <container-name> /bin/bash
```

1.6 Inspect container details (IP, config, volumes)

```
docker inspect <container-name>
```

1.7 Stop a container

```
docker stop <container-name>
```

1.8 Start a stopped container

```
docker start <container-name>
```

1.9 Restart a container

```
docker restart <container-name>
```

1.10 Remove a container

```
docker rm <container-name>
```

1.11 Remove an image

```
docker rmi <image-name>
```

1.12 System cleanup

Remove unused containers, images, networks:

```
docker system prune
```

Remove EVERYTHING unused:

```
docker system prune -a
```

2. Docker Network

What is Docker Network?

Docker Network is a virtual networking system that allows containers to communicate with each other and with the external world.

Why do we use it?

- To allow containers to talk using **container names** (e.g., `mongo`, `node-app`).
- To connect multi-container applications (Node + MongoDB).
- To isolate containers for security.
- To expose container services to the outside world.

Types of Docker Networks

1. **Bridge Network (Default):** Used for most applications.
2. **Host Network:** Shares host machine network; no port mapping needed.
3. **None Network:** Completely disables networking.

Important Commands

- List networks:
`docker network ls`
- Create a custom network:
`docker network create <network-name>`
- Inspect a network:
`docker network inspect <network-name>`
- Delete a network:
`docker network rm <network-name>`
- Run container inside a network:
`docker run --network <network-name> ...`
- Connect a running container to a network:
`docker network connect <network-name> <container>`

3. Docker Compose

What is Docker Compose?

Docker Compose is a tool used to run and manage multi-container applications using a single configuration file called `docker-compose.yml`.

Why do we use Docker Compose?

- To run multiple containers together (Node app + MongoDB, etc.).
- To avoid typing long `docker run` commands repeatedly.
- To define entire application setup in one YAML file.
- To easily start, stop, and rebuild complete environments.

Key Features

- Starts multiple services with one command.
- Automatically creates networks for communication.
- Supports volumes for data persistence.
- Simplifies development and deployment.

Important Commands

- Start all services:
`docker-compose up`
- Start in background:
`docker-compose up -d`
- Stop all services:
`docker-compose down`
- Rebuild containers:
`docker-compose up --build`
- View logs:
`docker-compose logs`

- Check running services:
- `docker-compose ps`

Basic Structure of docker-compose.yml

```
services:
  service-name:
    image / build
    ports
    environment
    volumes
    depends_on
volumes:
networks:
```

4. Setting Up Mongo & Mongo-Express

What is Mongo?

MongoDB is a NoSQL database used to store application data.

What is Mongo-Express?

Mongo-Express is a web-based GUI used to view and manage MongoDB databases easily.

Why do we use them together?

- MongoDB stores the actual application data.
- Mongo-Express lets us visually inspect collections, documents, and perform CRUD operations.
- Both services run as separate containers and communicate through a Docker network.

Requirements

To run them together, we need: - A custom Docker network - Mongo container - Mongo-Express container

Steps to Set Up

1. **Create a network** (so containers can communicate):
2. `docker network create mynetwork`
3. **Run MongoDB container:**
4. Assign a container name (easier communication).
5. Connect it to the same network.

```
Example: docker run -d --name mongo --network mynetwork -e
MONGO_INITDB_ROOT_USERNAME=<username> -e MONGO_INITDB_ROOT_PASSWORD=<password>
mongo
```

1. **Run Mongo-Express container:**
2. Connect to MongoDB through its container name.
3. Put both in the same network.

```
Example: docker run -d --name mongo-express --network mynetwork -e
ME_CONFIG_MONGODB_ADMINUSERNAME=<username> -e
ME_CONFIG_MONGODB_ADMINPASSWORD=<password> -e ME_CONFIG_MONGODB_SERVER=mongo -p
8081:8081 mongo-express
```

How They Communicate

- Mongo-Express connects to MongoDB using the name **mongo** (because they share the same network).
- No need to use IP addresses.

5. Connecting Database with Node App

Why do we connect Node App to MongoDB?

To store data like users, products, tasks, logs, etc.

How Node App Connects to MongoDB

- Both Node app and MongoDB must be in the same Docker network.
- Node app uses the MongoDB container name as the hostname.
- We define the DB connection string using environment variables.

Common Connection String

```
mongodb://<username>:<password>@mongo:27017
```

Here: - **mongo** → MongoDB container name - **27017** → default MongoDB port

Steps to Connect

1. Ensure Node app is also running inside the same network.
2. Provide the DB URL as an environment variable when running the Node container.

Example env variable: `MONGO_URL=mongodb://user:pass@mongo:27017`

Why This Works

- All containers communicate using container names.
- Docker handles DNS internally.
- No need for localhost or IP addresses.

Summary

- MongoDB stores data.
- Mongo-Express provides GUI.
- Node app connects to MongoDB through a shared Docker network.
- All components run as separate containers, but work together seamlessly.

6. Use Case: Dockerizing Our App

What does Dockerizing an app mean?

Dockerizing means packaging the entire application (code, runtime, dependencies) into a Docker image so it can run anywhere without setup.

Why do we Dockerize apps?

- Ensures the app runs the same on every machine.
- Removes dependency issues (Node versions, packages, OS differences).
- Makes deployment easier and faster.
- Allows running the app as a lightweight container.

Steps to Dockerize an App

1. Prepare your project folder (Node app, Python app, etc.).
 2. Create a **Dockerfile** with build instructions.
 3. Build the Docker image.
 4. Run the container.
 5. Push the image to Docker Hub.
-

7. Creating the Dockerfile

What is a Dockerfile?

A Dockerfile is a set of instructions that tells Docker how to build your image.

Why do we need it?

- Automates the environment setup.
- Defines OS, dependencies, and app commands.
- Ensures consistent builds.

Common Structure of a Dockerfile (Node App Example)

- Select base image (Node runtime)
- Set working directory
- Copy project files
- Install dependencies
- Expose port
- Define start command

Example (Simple Node App)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Why Alpine?

- Extremely lightweight
 - Faster build and push
-

8. Publishing the Image on Docker Hub

Why publish your image?

- To store your image online.
- To pull and run it on any machine.
- For deployment to cloud platforms.

Steps to Publish

1. Login to Docker Hub

```
docker login
```

2. Build Image

```
docker build -t myapp .
```

3. Tag Image

Tag format:

```
<username>/<repo>:tag
```

Example:

```
docker tag myapp satyajit86291/myapp:latest
```

4. Push Image

```
docker push satyajit86291/myapp:latest
```

After Push

- Image is stored permanently on Docker Hub.
- Anyone can pull it using:

```
docker pull satyajit86291/myapp:latest
```

Summary

- Dockerizing prepares your app to run anywhere.
- Dockerfile defines how your image is built.
- Publishing the image stores it safely, allows reuse, sharing, and deployment.

9. Docker Volumes

What are Docker Volumes?

Docker Volumes are used to store data **permanently** so that the data does not disappear when a container stops, restarts, or gets deleted.

Why do we use Volumes?

- Containers are **temporary** by nature.
- All data inside a container is lost when the container is removed.
- Volumes keep the data **safe and reusable**.
- Very important for databases (MongoDB, MySQL) or logs.

Simple Explanation (Shradha Mam Style)

- Container data = temporary
- Volume data = permanent
- Even if container is deleted, data stays safe.
- Like storing your project on GitHub so you can use it in future.

Types of Volumes

1. **Named Volume** – Managed by Docker (recommended).
 2. **Bind Mount** – Uses a folder from your PC (useful in development).
 3. **Anonymous Volume** – Automatically created; usually not used.
-

10. Volume Commands

Create a Volume

```
docker volume create myvol
```

Creates a permanent storage unit called `myvol`.

List All Volumes

```
docker volume ls
```

Shows all volumes on your system.

Inspect a Volume (Check Path)

```
docker volume inspect myvol
```

Shows where Docker is storing the data on your computer.

Remove a Volume

```
docker volume rm myvol
```

Deletes the volume (data will be lost).

Remove Unused Volumes

```
docker volume prune
```

Deletes all volumes that are not used by any container.

Using Volumes with Containers

1. Named Volume Example (MongoDB)

```
docker run -d
  --name mongo
  -v mongo-data:/data/db
  mongo
```

- `mongo-data` = volume name - `/data/db` = MongoDB storage folder inside container

If container is deleted, data stays safe in `mongo-data` volume.

2. Bind Mount Example (PC Folder ↔ Container)

```
docker run -it -v C:/Users/satya/Desktop/data:/appdata ubuntu
```

- Left side = your Windows path - Right side = folder inside Ubuntu container

Anything saved inside `/appdata` in Ubuntu appears in your Desktop folder.

Summary of Volumes

- Volumes store data permanently.
- Used mostly for databases and persistent files.
- Even after removing containers, data remains safe.
- Commands help create, inspect, and manage volumes.
- Bind mounts allow real-time syncing between host and container.